Shopping

> The latest version of Python you should use in this course is Python 3.12.

Write an AI to predict whether online shopping customers will complete a purchase.

```
$ python shopping.py shopping.csv
Correct: 4088
Incorrect: 844
True Positive Rate: 41.02%
True Negative Rate: 90.55%
```

When to Do It

By Wednesday, June 30, 2027 at 8:59 PM GMT-3

How to Get Help

1. Ask questions via Ed!
2. Ask questions via any of CS50's communities!

Background

When users are shopping online, not all will end up purchasing something. Most visitors to an online shopping website, in fact, likely don't end up going through with a purchase during that web browsing session. It might be useful, though, for a shopping website to be able to predict whether a user intends to make a purchase or not: perhaps displaying different content to the user, like showing the user a discount offer if the website believes the user isn't planning to complete the purchase. How could a website determine a user's purchasing intent? That's where machine learning will come in.

Your task in this problem is to build a nearest-neighbor classifier to solve this problem. Given information about a user — how many pages they've visited, whether they're shopping on a weekend, what web browser they're using, etc. — your classifier will predict whether or not the user will make a purchase. Your classifier won't be perfectly accurate — perfectly modeling human behavior is a task well beyond the scope of this class — but it should be better than guessing randomly. To train your classifier, we'll provide you with some data from a shopping website from about 12,000 users sessions.

How do we measure the accuracy of a system like this? If we have a testing data set, we could run our classifier on the data, and compute what proportion of the time we correctly classify the user's intent. This would give us a single accuracy percentage. But that number might be a little misleading. Imagine, for example, if about 15% of all users end up going through with a purchase. A classifier that always predicted

that the user would not go through with a purchase, then, we would measure as being 85% accurate: the only users it classifies incorrectly are the 15% of users who do go through with a purchase. And while 85% accuracy sounds pretty good, that doesn't seem like a very useful classifier.

Instead, we'll measure two values: sensitivity (also known as the "true positive rate") and specificity (also known as the "true negative rate"). Sensitivity refers to the proportion of positive examples that were correctly identified: in other words, the proportion of users who did go through with a purchase who were correctly identified. Specificity refers to the proportion of negative examples that were correctly identified: in this case, the proportion of users who did not go through with a purchase who were correctly identified. So our "always guess no" classifier from before would have perfect specificity (1.0) but no sensitivity (0.0). Our goal is to build a classifier that performs reasonably on both metrics.

Getting Started

Download the distribution code from https://cdn.cs50.net/ai/2023/x/projects/4/shopping.zip and unzip it.

Run `pip3 install scikit-learn` to install the `scikit-learn` package if it isn't already installed, which you'll need for this project.

Understanding

First, open up `shopping.csv`, the data set provided to you for this project. You can open it in a text editor, but you may find it easier to understand visually in a spreadsheet application like Microsoft Excel, Apple Numbers, or Google Sheets.

There are about 12,000 user sessions represented in this spreadsheet: represented as one row for each user session. The first six columns measure the different types of pages users have visited in the session: the `Administrative`, `Informational`, and `ProductRelated` columns measure how many of those types of pages the user visited, and their corresponding `_Duration` columns measure how much time the user spent on any of those pages. The `BounceRates`, `ExitRates`, and `PageValues` columns measure information from Google Analytics about the page the user visited. `SpecialDay` is a value that measures how close the date of the user's session is to a special day (like Valentine's Day or Mother's Day). `Month` is an abbreviation of the month the user visited. `OperatingSystems`, `Browser`, `Region`, and `TrafficType` are all integers describing information about the user themself. `VisitorType` will take on the value `Returning_Visitor` for returning visitors and some other string value for non-returning visitors. `Weekend` is `TRUE` or `FALSE` depending on whether or not the user is visiting on a weekend.

Perhaps the most important column, though, is the last one: the `Revenue` column. This is the column that indicates whether the user ultimately made a purchase or not: `TRUE` if they did, `FALSE` if they didn't. This is the column that we'd like to learn to predict (the "label"), based on the values for all of the other columns (the "evidence").

Next, take a look at `shopping.py`. The `main` function loads data from a CSV spreadsheet by calling the `load_data` function and splits the data into a training and testing set. The `train_model` function is then called to train a machine learning model on the training data. Then, the model is used to make

predictions on the testing data set. Finally, the `evaluate` function determines the sensitivity and specificity of the model, before the results are ultimately printed to the terminal.

The functions `load_data`, `train_model`, and `evaluate` are left blank. That's where you come in!

Specification

Complete the implementation of `load_data`, `train_model`, and `evaluate` in `shopping.py`.

The `load_data` function should accept a CSV filename as its argument, open that file, and return a tuple `(evidence, labels)`. `evidence` should be a list of all of the evidence for each of the data points, and `labels` should be a list of all of the labels for each data point.

- Since you'll have one piece of evidence and one label for each row of the spreadsheet, the length of the `evidence` list and the length of the `labels` list should ultimately be equal to the number of rows in the CSV spreadsheet (excluding the header row). The lists should be ordered according to the order the users appear in the spreadsheet. That is to say, `evidence[0]` should be the evidence for the first user, and `labels[0]` should be the label for the first user.
- Each element in the `evidence` list should itself be a list. The list should be of length 17: the number of columns in the spreadsheet excluding the final column (the label column).
- The values in each `evidence` list should be in the same order as the columns that appear in the evidence spreadsheet. You may assume that the order of columns in `shopping.csv` will always be presented in that order.
- Note that, to build a nearest-neighbor classifier, all of our data needs to be numeric. Be sure that your values have the following types:
  - `Administrative`, `Informational`, `ProductRelated`, `Month`, `OperatingSystems`, `Browser`, `Region`, `TrafficType`, `VisitorType`, and `Weekend` should all be of type `int`
  - `Administrative_Duration`, `Informational_Duration`, `ProductRelated_Duration`, `BounceRates`, `ExitRates`, `PageValues`, and `SpecialDay` should all be of type `float`.
  - `Month` should be `0` for January, `1` for February, `2` for March, etc. up to `11` for December.
  - `VisitorType` should be `1` for returning visitors and `0` for non-returning visitors.
  - `Weekend` should be `1` if the user visited on a weekend and `0` otherwise.
- Each value of `labels` should either be the integer `1`, if the user did go through with a purchase, or `0` otherwise.
- For example, the value of the first evidence list should be `[0, 0.0, 0, 0.0, 1, 0.0, 0.2, 0.2, 0.0, 0.0, 1, 1, 1, 1, 1, 1, 0]` and the value of the first label should be `0`.

The `train_model` function should accept a list of evidence and a list of labels, and return a `scikit-learn` nearest-neighbor classifier (a k-nearest-neighbor classifier where `k = 1`) fitted on that training data.

- Notice that we've already imported for you `from sklearn.neighbors import KNeighborsClassifier`. You'll want to use a `KNeighborsClassifier` in this function.

The `evaluate` function should accept a list of `labels` (the true labels for the users in the testing set) and a list of `predictions` (the labels predicted by your classifier), and return two floating-point values

`(sensitivity, specificity)`.

- `sensitivity` should be a floating-point value from 0 to 1 representing the "true positive rate": the proportion of actual positive labels that were accurately identified.
- `specificity` should be a floating-point value from 0 to 1 representing the "true negative rate": the proportion of actual negative labels that were accurately identified.
- You may assume each label will be `1` for positive results (users who did go through with a purchase) or `0` for negative results (users who did not go through with a purchase).
- You may assume that the list of true labels will contain at least one positive label and at least one negative label.

You should not modify anything else in `shopping.py` other than the functions the specification calls for you to implement, though you may write additional functions and/or import other Python standard library modules. You may also import `numpy` or `pandas` or anything from `scikit-learn`, if familiar with them, but you should not use any other third-party Python modules. You should not modify `shopping.csv`.

Hints

- For information and examples about how to load data from a CSV file, see Python's CSV documentation.

Testing

If you'd like, you can execute the below (after setting up `check50` on your system) to evaluate the correctness of your code. This isn't obligatory; you can simply submit following the steps at the end of this specification, and these same tests will run on our server. Either way, be sure to compile and test it yourself as well!

```
check50 ai50/projects/2024/x/shopping
```

Execute the below to evaluate the style of your code using `style50`.

```
style50 shopping.py
```

Remember that **you may not import any modules** (other than those in the Python standard library) **other than those explicitly authorized herein**. Doing so will not only prevent `check50` from running, but will also prevent `submit50` from scoring your assignment, since it uses `check50`. If that happens, you've likely imported something disallowed or otherwise modified the distribution code in an unauthorized manner, per the specification. There are certainly tools out there that trivialize some of these projects, but that's not the goal here; you're learning things at a lower level. If we don't say here that you can use them, you can't use them.

How to Submit

1. Visit this link, log in with your GitHub account, and click **Authorize cs50**. Then, check the box indicating that you'd like to grant course staff access to your submissions, and click **Join course**.
2. Install Git and, optionally, install `submit50`.
3. If you've installed `submit50`, execute

   ```
   submit50 ai50/projects/2024/x/shopping
   ```

   Otherwise, using Git, push your work to `https://github.com/me50/USERNAME.git`, where `USERNAME` is your GitHub username, on a branch called `ai50/projects/2024/x/shopping`.

If you submit your code directly using Git, rather than `submit50`, **do not** include files or folders other than those you are actually instructed to modify in the specification above. (That is to say, don't upload your entire directory!)

Work should be graded within five minutes. You can then go to https://cs50.me/cs50ai to view your current progress!

Acknowledgements

Data set provided by Sakar, C.O., Polat, S.O., Katircioglu, M. et al. Neural Comput & Applic (2018)